

Trabalho de Laboratório — Padrão Factory Method em Java

Sistema de pedidos de pagamento de veículos

Objetivo

Modelar, em Java, a arquitetura de um sistema simples utilizando o padrão Factory Method.

O objetivo é compreender como a criação de objetos pode ser delegada às subclasses, evitando que o programa principal crie diretamente os produtos concretos.

A implementação em Java será apresentada durante o laboratório.

Entrega

- Diagrama UML.
- Questões de fixação.
- Não entregar código Java.

Contexto

Considere um sistema de venda de veículos.

O sistema permite que diferentes tipos de clientes realizem pedidos de compra.

Existem dois tipos de clientes:

- Cliente à vista.
- Cliente por crédito.

Cada tipo de cliente deve criar um tipo específico de pedido.

A criação do pedido não deve ser feita diretamente pelo programa principal, mas delegada às subclasses de Cliente.

Estrutura esperada

Criador abstrato

Cliente

Criadores concretos

ClienteAVista

ClienteCredito

Produto abstrato

Pedido

Produtos concretos

PedidoAVista

PedidoCredito

Parte 1 — Classe abstrata Cliente

Representar no diagrama a classe abstrata Cliente.

Atributo

pedidos : List<Pedido>

Métodos

novoPedido(valor : double) : void

criarPedido(valor : double) : Pedido

Observações

- criarPedido deve ser abstrato.
- novoPedido deve ser concreto.
- criarPedido representa o Factory Method.
- novoPedido define o algoritmo geral para criar, validar, pagar e armazenar um pedido.

Parte 2 — Criadores concretos

ClienteAVista

- Herda de Cliente.
- Implementa:

criarPedido(valor : double) : Pedido

Esse método deve criar um PedidoAVista.

ClienteCredito

- Herda de Cliente.
- Implementa:

criarPedido(valor : double) : Pedido

Esse método deve criar um PedidoCredito.

Parte 3 — Produto abstrato Pedido

Representar no diagrama a classe abstrata Pedido.

Atributo

valor : double

Métodos

valido() : boolean

pagar() : void

Observação

- Ambos os métodos são abstratos.

Parte 4 — Produtos concretos

PedidoAVista

- Herda de Pedido.
- Implementa:

valido() : boolean

pagar() : void

A validação deve sempre retornar verdadeiro.

PedidoCredito

- Herda de Pedido.
- Implementa:

valido() : boolean

pagar() : void

A validação deve aceitar apenas pedidos com valor entre 1000 e 5000.

Relações obrigatórias

Representar claramente no diagrama:

- ClienteAVista herda de Cliente.
- ClienteCredito herda de Cliente.
- PedidoAVista herda de Pedido.
- PedidoCredito herda de Pedido.
- Cliente possui uma lista de Pedido.
- ClienteAVista cria PedidoAVista.
- ClienteCredito cria PedidoCredito.

Entrega

O aluno deverá entregar:

1. O diagrama UML completo.

O diagrama deve representar apenas a arquitetura do sistema.

2. As respostas das questões de fixação.

Não é necessário entregar código Java.

Restrições

- Não incluir classe de teste.
- Não incluir código Java.
- Representar apenas a arquitetura.
- Usar nomes em português.
- O diagrama deve respeitar o código apresentado em aula.
- A classe Usuario não deve aparecer no diagrama.

Cenário de referência

Cliente cliente;

```
cliente = new ClienteAVista();  
cliente.novoPedido(2000.0);  
cliente.novoPedido(10000.0);
```

```
cliente = new ClienteCredito();  
cliente.novoPedido(2000.0);  
cliente.novoPedido(10000.0);
```

Saída esperada

Pagamento do pedido à vista de valor 2000.0 realizado.

Pagamento do pedido à vista de valor 10000.0 realizado.

Pagamento do pedido por crédito de valor 2000.0 realizado.

O pedido por crédito no valor de 10000.0 é recusado, pois não passa na validação.

Perguntas adicionais

1. Qual classe representa o criador abstrato?

2. Quais classes representam os criadores concretos?
3. Qual classe representa o produto abstrato?
4. Quais classes representam os produtos concretos?
5. Qual método representa o Factory Method?
6. Por que criarPedido() deve ser abstrato?
7. Por que novoPedido() deve ser concreto?
8. Qual classe decide criar um PedidoAVista?
9. Qual classe decide criar um PedidoCredito?
10. Por que o programa principal pode trabalhar apenas com o tipo abstrato Cliente?
11. Qual é a vantagem de Cliente manipular objetos do tipo Pedido em vez de classes concretas?
12. Por que este projeto é um exemplo do padrão Factory Method?
13. Qual é a diferença principal entre Factory Method e Simple Factory?
14. O que aconteceria se a criação dos pedidos fosse feita diretamente no programa principal?
15. Como o sistema poderia ser estendido para incluir um novo tipo de cliente, por exemplo ClienteFinanciamento?

Interpretação

- Cliente define o algoritmo geral para criar e processar um pedido.
- criarPedido é o Factory Method.
- Cada subclasse de Cliente decide qual tipo concreto de Pedido será criado.
- O programa principal trabalha apenas com o tipo abstrato Cliente, sem precisar conhecer as classes concretas de Pedido.

Objetivo final

Construir um diagrama UML que permita delegar a criação dos pedidos às subclasses de Cliente, mantendo a estrutura geral do sistema baseada em abstrações.

O código Java será apresentado e desenvolvido durante o laboratório.